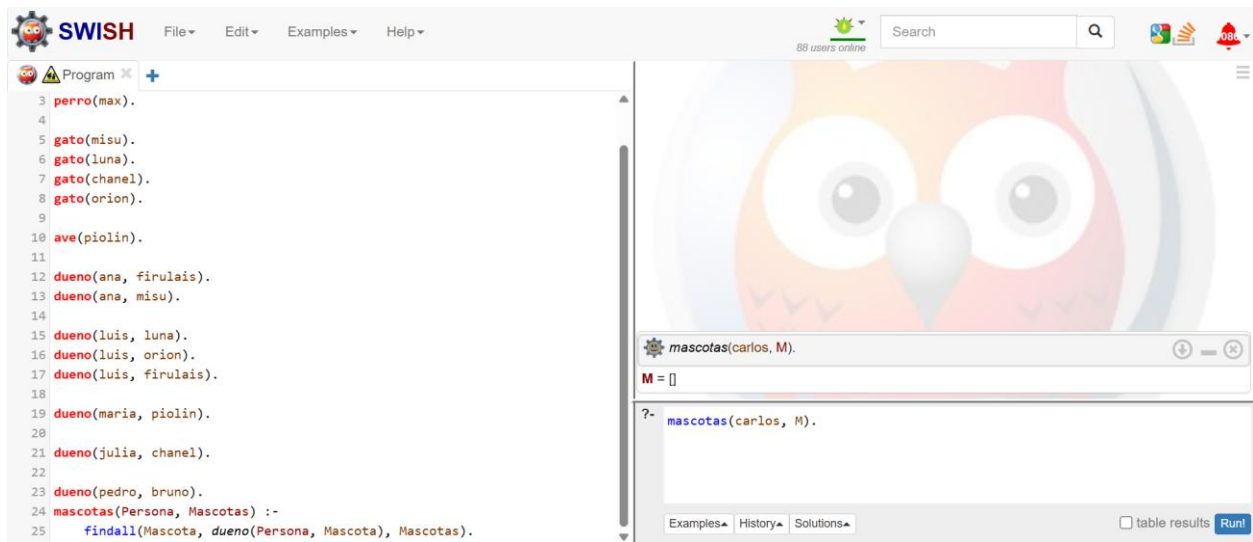


1.

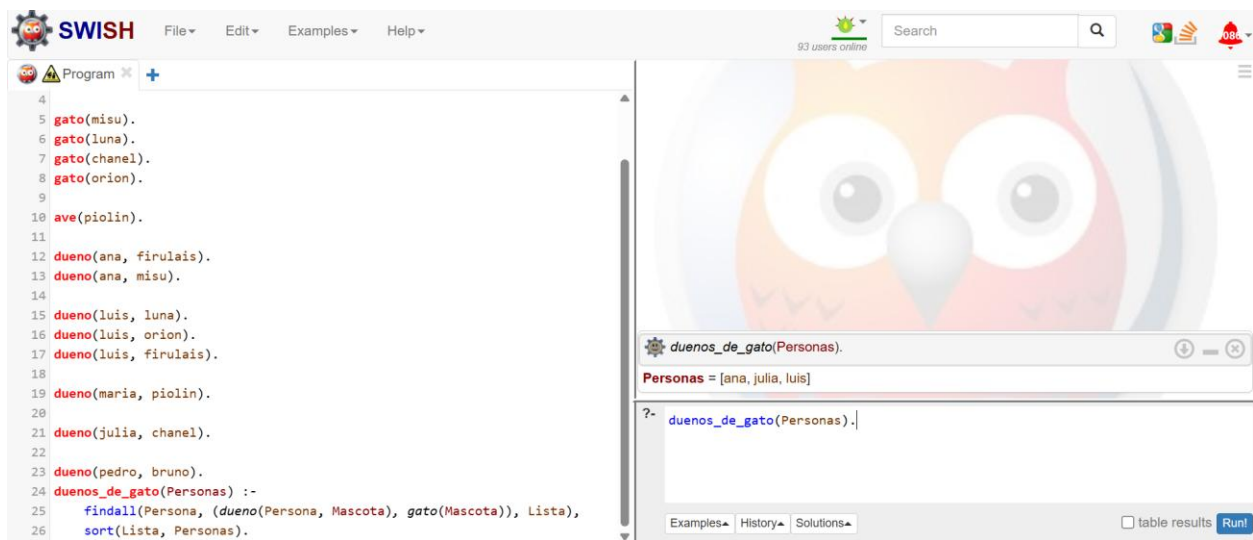


The screenshot shows the SWISH IDE interface. The left pane contains a Prolog program with the following code:

```
3 perro(max).
4
5 gato(misu).
6 gato(luna).
7 gato(chanel).
8 gato(orion).
9
10 ave(piolin).
11
12 dueno(ana, firulais).
13 dueno(ana, misu).
14
15 dueno(luis, luna).
16 dueno(luis, orion).
17 dueno(luis, firulais).
18
19 dueno(maria, piolin).
20
21 dueno(julia, chanel).
22
23 dueno(pedro, bruno).
24 mascotas(Persona, Mascotas) :-
25     findall(Mascota, dueno(Persona, Mascota), Mascotas).
```

The right pane shows a query window with the query `mascotas(carlos, M).` and the result `M = []`. The background features a large owl illustration.

2.



The screenshot shows the SWISH IDE interface. The left pane contains a Prolog program with the following code:

```
4
5 gato(misu).
6 gato(luna).
7 gato(chanel).
8 gato(orion).
9
10 ave(piolin).
11
12 dueno(ana, firulais).
13 dueno(ana, misu).
14
15 dueno(luis, luna).
16 dueno(luis, orion).
17 dueno(luis, firulais).
18
19 dueno(maria, piolin).
20
21 dueno(julia, chanel).
22
23 dueno(pedro, bruno).
24 duenos_de_gato(Personas) :-
25     findall(Persona, (dueno(Persona, Mascota), gato(Mascota)), Lista),
26     sort(Lista, Personas).
```

The right pane shows a query window with the query `duenos_de_gato(Personas).` and the result `Personas = [ana, julia, luis]`. The background features a large owl illustration.

3.

The screenshot shows the SWISH Prolog IDE interface. The left pane contains the following Prolog code:

```

5.
6.
7. l).
8. ).
9.
10. ).
11.
12. firulais).
13. misu).
14.
15. luna).
16. orion).
17. firulais).
18.
19. a, piolin).
20.
21. a, chanel).
22.
23. o, bruno).
24.
25. cos_de_gato(Tipo, Personas) :-
26. Persona, Mascota^(dueno(Persona, Mascota), call(Tipo, Mascota)), Personas).

```

The right pane shows the execution results for the query `duenos_unicos_de_gato(gato, Personas).`. The results are:

```

Personas = [ana, julia, luis]
?- duenos_unicos_de_gato(gato, Personas).

```

The interface also shows a search bar, a user count of 107 users online, and a 'Run!' button.

4.

The screenshot shows the SWISH Prolog IDE interface. The left pane contains the following Prolog code:

```

8. gato(orion).
9.
10. ave(piolin).
11.
12. dueno(ana, firulais).
13. dueno(ana, misu).
14.
15. dueno(luis, luna).
16. dueno(luis, orion).
17. dueno(luis, firulais).
18.
19. dueno(maria, piolin).
20.
21. dueno(julia, chanel).
22.
23. dueno(pedro, bruno).
24.
25. tipo_mascota(Mascota, perro) :- perro(Mascota).
26. tipo_mascota(Mascota, gato) :- gato(Mascota).
27. tipo_mascota(Mascota, ave) :- ave(Mascota).
28. tipo_mascota(Persona, Tipos) :-
29. setof(Tipo, Mascota^(dueno(Persona, Mascota), tipo_mascota(Mascota, Tipo)), Tipos).

```

The right pane shows the execution results for the query `tipos_mascota(ana, Tipos).`. The results are:

```

Tipos = [gato, perro]
tipos_mascota(maria, Tipos).
Tipos = [ave]
?- tipos_mascota(maria, Tipos).

```

The interface also shows a search bar, a user count of 117 users online, and a 'Run!' button.

5.

1. ¿Por qué bagof/3 genera varias soluciones?

Porque en el objetivo hay una variable, Persona, que no aparece en la plantilla. Entonces bagof no junta todo en una sola lista, sino que va generando una lista distinta por cada persona, y toca usar el punto y coma para ver la siguiente.

2. ¿Qué variable está siendo utilizada para realizar la agrupación?

La variable Persona, porque es la que queda libre en la consulta y no está en la plantilla, entonces Prolog la usa para separar los resultados por grupos.

3. ¿Qué ocurriría si se utilizara findall/3?

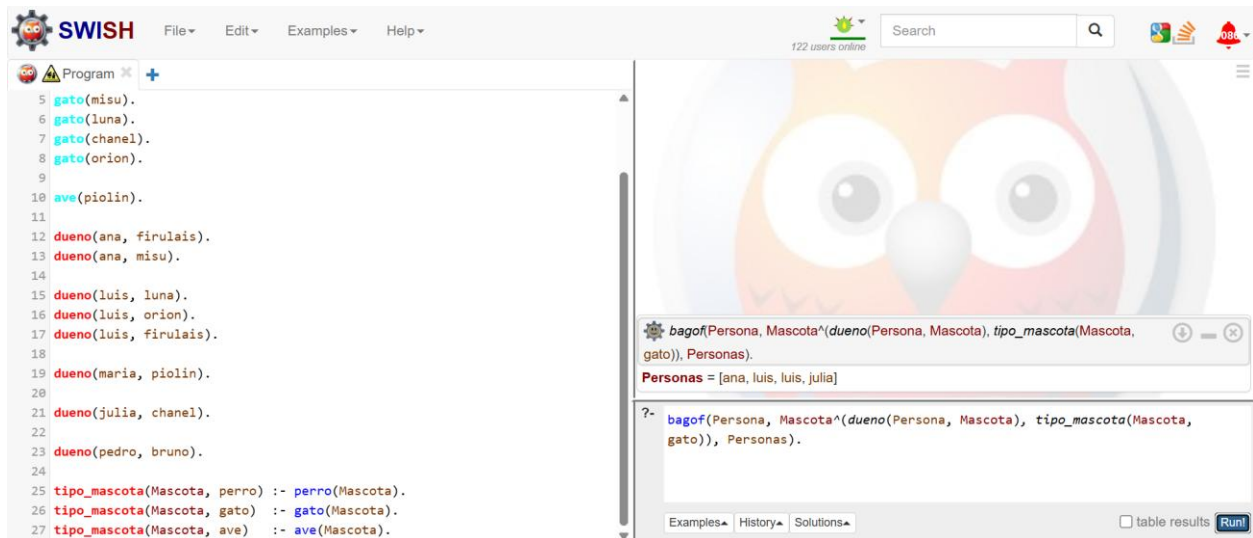
Con findall todas las mascotas quedarían mezcladas en una sola lista, sin importar de quién son, porque findall no agrupa por Persona, simplemente junta todas las soluciones que encuentra.

4. ¿Qué ocurriría si se utilizara setof/3?

Pasaría casi lo mismo que con bagof, o sea que también agrupa por Persona y da una solución a la vez, pero la diferencia es que setof ordena tanto las mascotas dentro de cada grupo como el orden en que van saliendo las personas.

6.

1)



The screenshot shows the SWISH Prolog IDE. The code editor on the left contains the following Prolog code:

```
5 gato(misu).
6 gato(luna).
7 gato(chanel).
8 gato(orion).
9
10 ave(piolin).
11
12 dueno(ana, firulais).
13 dueno(ana, misu).
14
15 dueno(luis, luna).
16 dueno(luis, orion).
17 dueno(luis, firulais).
18
19 dueno(maria, piolin).
20
21 dueno(julia, chanel).
22
23 dueno(pedro, bruno).
24
25 tipo_mascota(Mascota, perro) :- perro(Mascota).
26 tipo_mascota(Mascota, gato) :- gato(Mascota).
27 tipo_mascota(Mascota, ave) :- ave(Mascota).
```

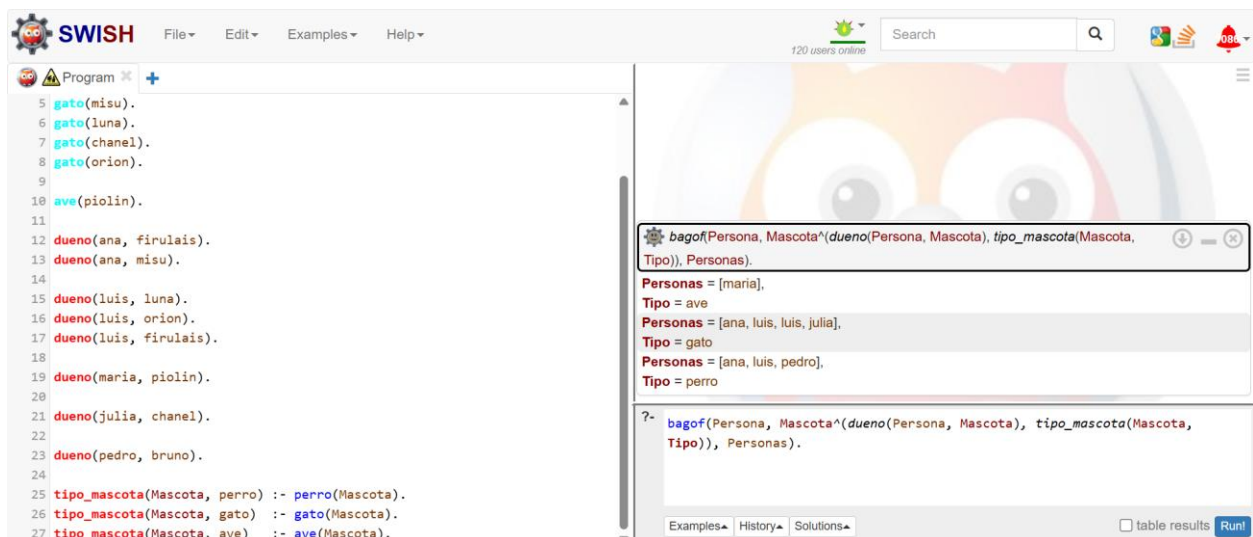
The right pane shows the execution of the query:

```
bagof(Persona, Mascota^(dueno(Persona, Mascota), tipo_mascota(Mascota, gato)), Personas).
```

The result is:

```
Personas = [ana, luis, luis, julia]
```

2)



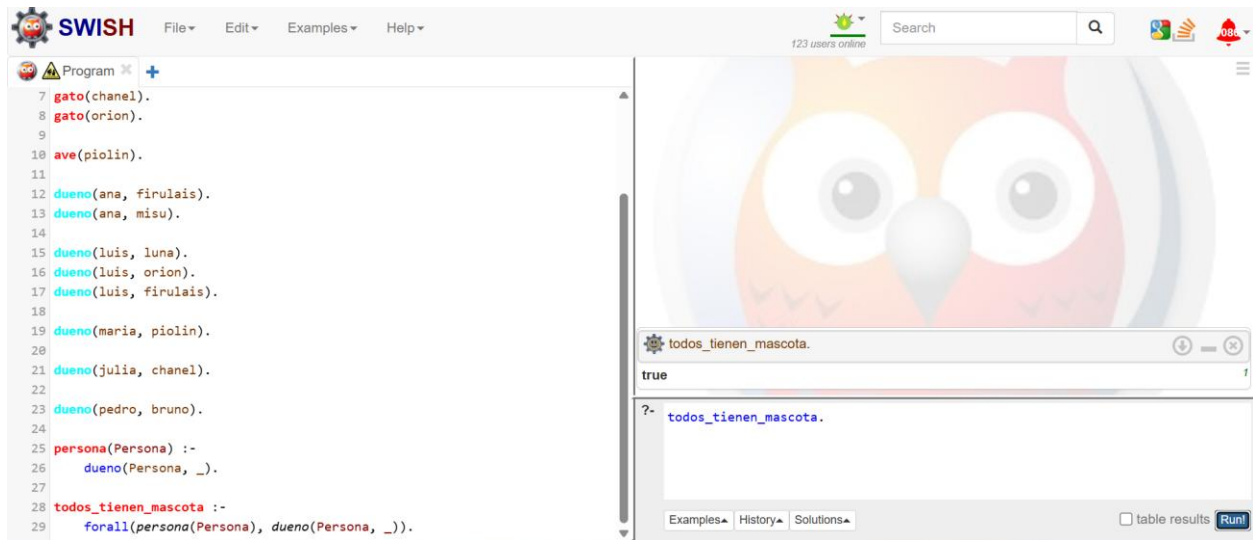
The screenshot shows the SWISH Prolog IDE with the same code as the previous one. The right pane shows the execution of the query:

```
setof(Persona, Mascota^(dueno(Persona, Mascota), tipo_mascota(Mascota, Tipo)), Personas).
```

The result is:

```
Personas = [maria],
Tipo = ave
Personas = [ana, luis, luis, julia],
Tipo = gato
Personas = [ana, luis, pedro],
Tipo = perro
```

7.



The screenshot shows the SWISH Prolog IDE interface. The left pane contains a Prolog program with the following code:

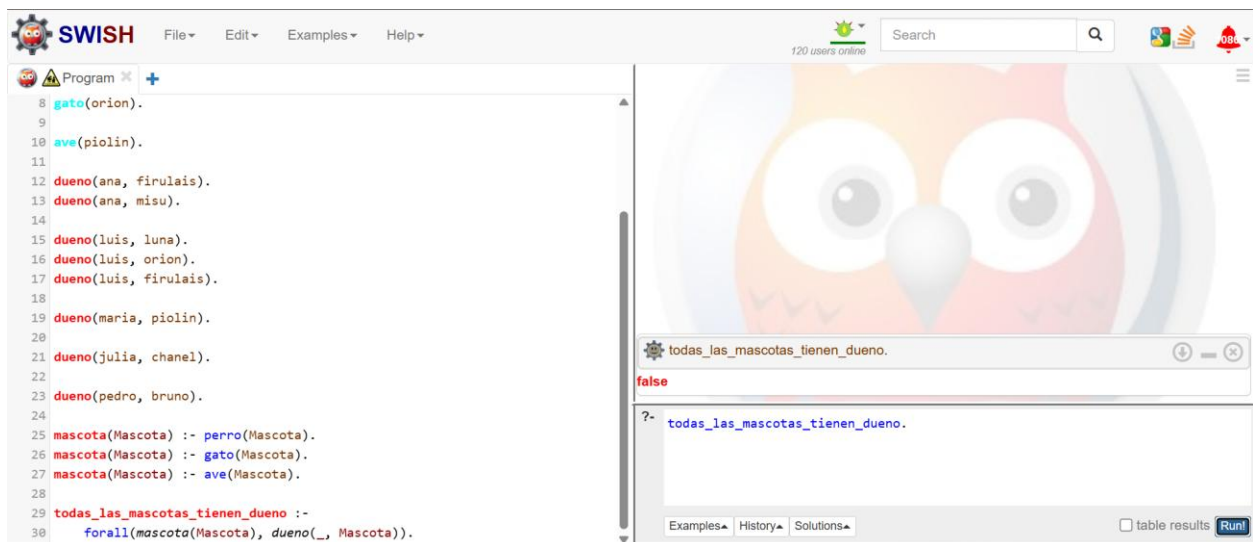
```

7 gato(chanel).
8 gato(orion).
9
10 ave(piolin).
11
12 dueno(ana, firulais).
13 dueno(ana, misu).
14
15 dueno(luis, luna).
16 dueno(luis, orion).
17 dueno(luis, firulais).
18
19 dueno(maria, piolin).
20
21 dueno(julia, chanel).
22
23 dueno(pedro, bruno).
24
25 persona(Persona) :-
26     dueno(Persona, _).
27
28 todos_tienen_mascota :-
29     forall(persona(Persona), dueno(Persona, _)).

```

The right pane shows the execution results. The query `todos_tienen_mascota.` has been executed, and the result is `true`. The background features a large, stylized owl illustration.

8.



The screenshot shows the SWISH Prolog IDE interface. The left pane contains a Prolog program with the following code:

```

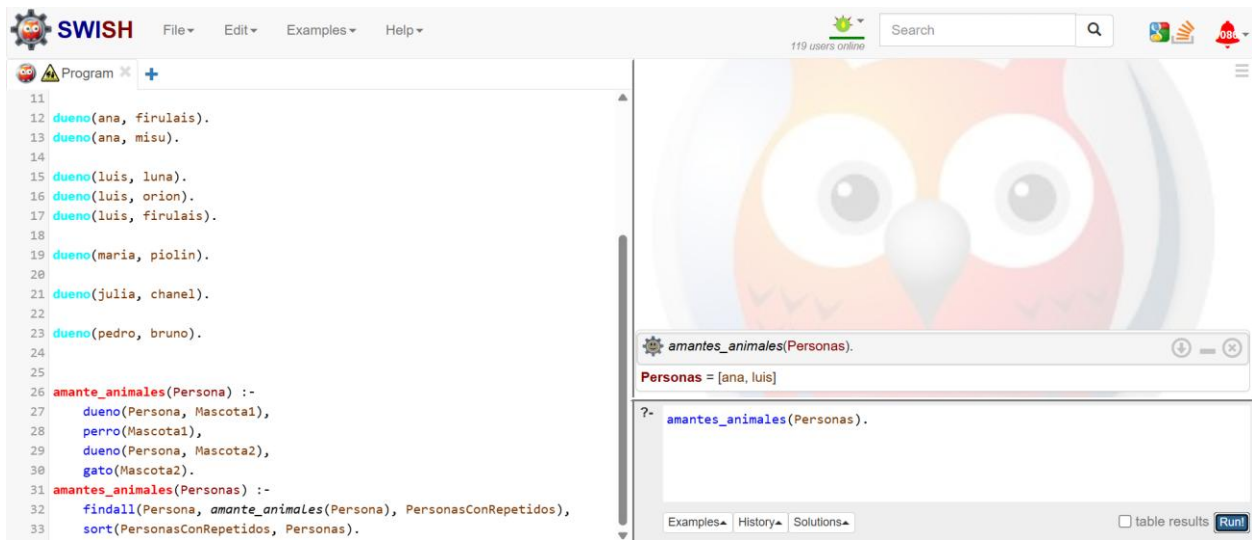
8 gato(orion).
9
10 ave(piolin).
11
12 dueno(ana, firulais).
13 dueno(ana, misu).
14
15 dueno(luis, luna).
16 dueno(luis, orion).
17 dueno(luis, firulais).
18
19 dueno(maria, piolin).
20
21 dueno(julia, chanel).
22
23 dueno(pedro, bruno).
24
25 mascota(Mascota) :- perro(Mascota).
26 mascota(Mascota) :- gato(Mascota).
27 mascota(Mascota) :- ave(Mascota).
28
29 todas_las_mascotas_tienen_dueno :-
30     forall(mascota(Mascota), dueno(_, Mascota)).

```

The right pane shows the execution results. The query `todas_las_mascotas_tienen_dueno.` has been executed, and the result is `false`. The background features a large, stylized owl illustration.

9.

Reto1



The image shows the SWISH IDE interface for Reto1. The left pane displays a Prolog program with the following code:

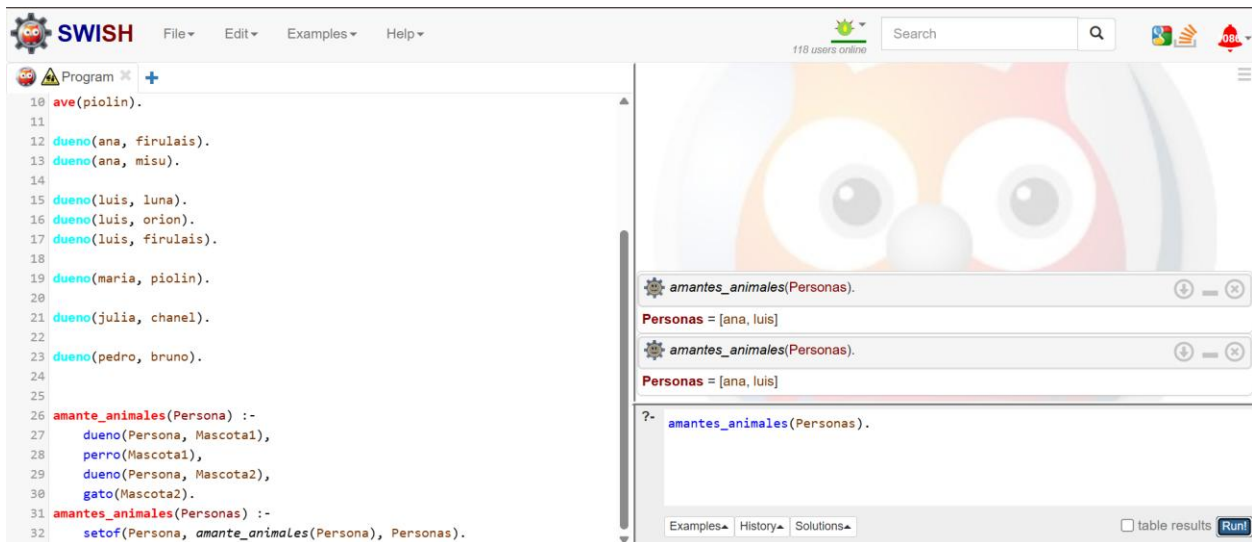
```
11
12 dueno(ana, firulais).
13 dueno(ana, misu).
14
15 dueno(luis, luna).
16 dueno(luis, orion).
17 dueno(luis, firulais).
18
19 dueno(maria, piolin).
20
21 dueno(julia, chanel).
22
23 dueno(pedro, bruno).
24
25
26 amante_animales(Persona) :-
27     dueno(Persona, Mascota1),
28     perro(Mascota1),
29     dueno(Persona, Mascota2),
30     gato(Mascota2).
31 amantes_animales(Personas) :-
32     findall(Persona, amante_animales(Persona), PersonasConRepetidos),
33     sort(PersonasConRepetidos, Personas).
```

The right pane shows the execution environment with a large owl graphic in the background. Below the graphic, there is a console window with the following content:

```
amantes_animales(Personas).
Personas = [ana, luis]
```

At the bottom of the right pane, there is a search bar and a "Run!" button.

Reto 2



The image shows the SWISH IDE interface for Reto 2. The left pane displays a Prolog program with the following code:

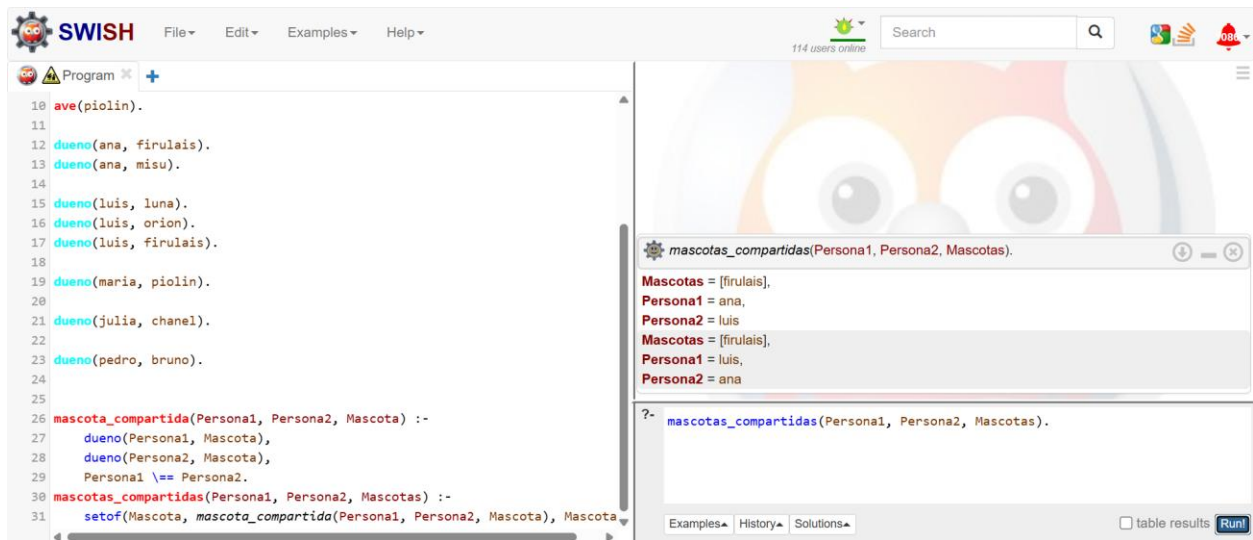
```
10 ave(piolin).
11
12 dueno(ana, firulais).
13 dueno(ana, misu).
14
15 dueno(luis, luna).
16 dueno(luis, orion).
17 dueno(luis, firulais).
18
19 dueno(maria, piolin).
20
21 dueno(julia, chanel).
22
23 dueno(pedro, bruno).
24
25
26 amante_animales(Persona) :-
27     dueno(Persona, Mascota1),
28     perro(Mascota1),
29     dueno(Persona, Mascota2),
30     gato(Mascota2).
31 amantes_animales(Personas) :-
32     setof(Persona, amante_animales(Persona), Personas).
```

The right pane shows the execution environment with a large owl graphic in the background. Below the graphic, there is a console window with the following content:

```
amantes_animales(Personas).
Personas = [ana, luis]
```

At the bottom of the right pane, there is a search bar and a "Run!" button.

10.



The SWISH IDE interface displays a Prolog program in the left pane and its execution results in the right pane. The program defines a list of pet owners and a predicate for shared pets.

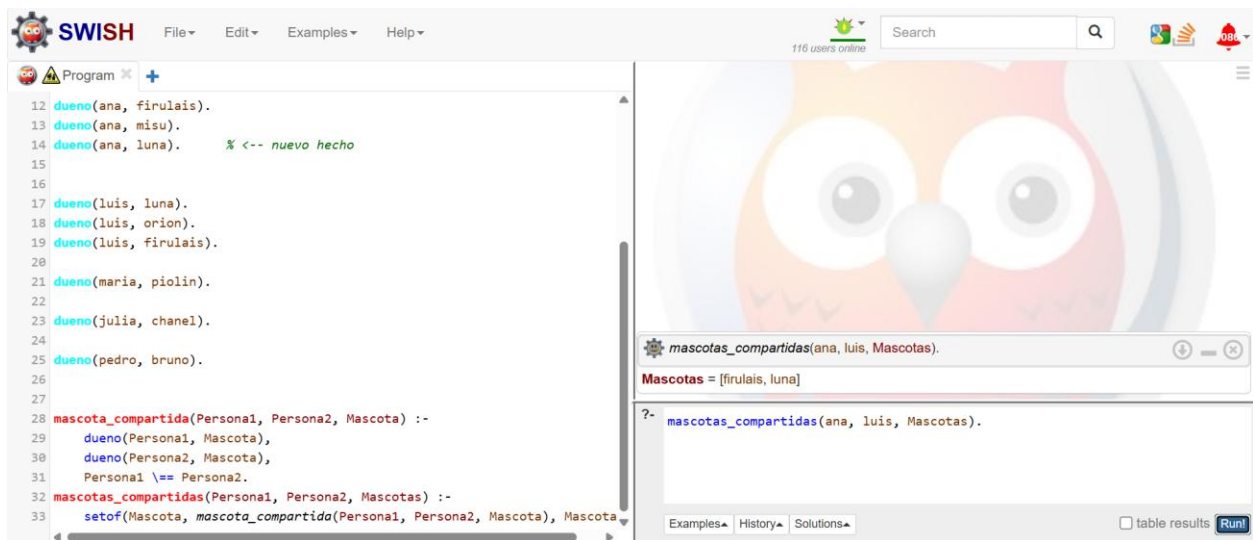
```
10 ave(piolin).
11
12 dueño(ana, firulais).
13 dueño(ana, misu).
14
15 dueño(luis, luna).
16 dueño(luis, orion).
17 dueño(luis, firulais).
18
19 dueño(maria, piolin).
20
21 dueño(julia, chanel).
22
23 dueño(pedro, bruno).
24
25
26 mascota_compartida(Persona1, Persona2, Mascota) :-
27     dueño(Persona1, Mascota),
28     dueño(Persona2, Mascota),
29     Persona1 \= Persona2.
30
31 mascotas_compartidas(Persona1, Persona2, Mascotas) :-
32     setof(Mascota, mascota_compartida(Persona1, Persona2, Mascota), Mascotas).
```

The right pane shows the execution of the query `mascotas_compartidas(Persona1, Persona2, Mascotas).`. The results are:

```
Mascotas = [firulais].
Persona1 = ana,
Persona2 = luis
Mascotas = [firulais].
Persona1 = luis,
Persona2 = ana
```

The bottom of the right pane shows the query being executed: `?- mascotas_compartidas(Persona1, Persona2, Mascotas).`

Caso adicional



The SWISH IDE interface displays a modified Prolog program in the left pane and its execution results in the right pane. The program defines a list of pet owners and a predicate for shared pets. A new fact is added to the list of owners.

```
12 dueño(ana, firulais).
13 dueño(ana, misu).
14 dueño(ana, luna). % <-- nuevo hecho
15
16
17 dueño(luis, luna).
18 dueño(luis, orion).
19 dueño(luis, firulais).
20
21 dueño(maria, piolin).
22
23 dueño(julia, chanel).
24
25 dueño(pedro, bruno).
26
27
28 mascota_compartida(Persona1, Persona2, Mascota) :-
29     dueño(Persona1, Mascota),
30     dueño(Persona2, Mascota),
31     Persona1 \= Persona2.
32
33 mascotas_compartidas(Persona1, Persona2, Mascotas) :-
34     setof(Mascota, mascota_compartida(Persona1, Persona2, Mascota), Mascotas).
```

The right pane shows the execution of the query `mascotas_compartidas(ana, luis, Mascotas).`. The results are:

```
Mascotas = [firulais, luna]
```

The bottom of the right pane shows the query being executed: `?- mascotas_compartidas(ana, luis, Mascotas).`

8. PROBLEMA DE LA RANA

The screenshot shows the SWISH IDE interface. On the left, a Prolog program is loaded in the 'Program' tab. The code defines a maximum jump distance, a function to calculate the next state, a depth-first search (dfs) function, and a search function (buscar_solucion). The right pane shows the execution results for the query `buscar_solucion(Solucion).`. The results indicate that a solution was found, displaying the sequence of states: `[pos(orilla_inicial), pos(piedra1), pos(piedra2), pos(piedra3), pos(orilla_final)]`. The interface includes a search bar, a user count (111 users online), and a 'Run!' button.

```
7 salto_maximo(4.0).
9
10
11 siguiente_estado(pos(LugarActual), pos(LugarSiguiente)) :-
12     ubicacion(LugarActual, X1, Y1),
13     ubicacion(LugarSiguiente, X2, Y2),
14     LugarActual \== LugarSiguiente,
15     salto_maximo(Max),
16     Distancia is sqrt((X2 - X1)**2 + (Y2 - Y1)**2),
17     Distancia <= Max.
18
19 dfs(pos(orilla_final), Visitados, Solucion) :-
20     reverse(Visitados, Solucion).
21
22 dfs(EstadoActual, Visitados, Solucion) :-
23     siguiente_estado(EstadoActual, EstadoSiguiente),
24     \+ member(EstadoSiguiente, Visitados),
25     dfs(EstadoSiguiente, [EstadoSiguiente | Visitados], Solucion).
26
27 buscar_solucion(Solucion) :-
28     EstadoInicial = pos(orilla_inicial),
29     dfs(EstadoInicial, [EstadoInicial], Solucion).
```

9. PROBLEMA DE BATMAN VS VILLANOS

The screenshot shows the SWISH IDE interface. On the left, a Prolog program is loaded in the 'Program' tab. The code defines a function to calculate the next state, a depth-first search (dfs) function, and a search function (batman_can_win). The right pane shows the execution results for the query `batman_can_win(60).`. The results indicate that a solution was found, displaying the sequence of states: `[pos(orilla_inicial), pos(piedra1), pos(piedra2), pos(piedra3), pos(orilla_final)]`. The interface includes a search bar, a user count (112 users online), and a 'Run!' button.

```
14 % --- REGLAS ---
15
16 siguiente_estado(estado(Villanos, Poderes, Energia), estado(VillanosRestante,
17     select(villain(Nombre, Salud, Debilidades), Villanos, VillanosRestantes
18     member(NombrePoder, Debilidades),
19     member(power(NombrePoder, Dano, Costo), Poderes),
20     Dano >= Salud,
21     Costo <= Energia,
22     EnergiaRestante is Energia - Costo.
23
24 dfs(estado([], _, _), _, _).
25
26 dfs(EstadoActual, Visitados) :-
27     siguiente_estado(EstadoActual, EstadoSiguiente),
28     \+ member(EstadoSiguiente, Visitados),
29     dfs(EstadoSiguiente, [EstadoSiguiente | Visitados]).
30
31 batman_can_win(EnergiaMaxima) :-
32     power_list(Superpoderes),
33     villain_list(Villanos),
34     EstadoInicial = estado(Villanos, Superpoderes, EnergiaMaxima),
35     dfs(EstadoInicial, [EstadoInicial]).
```

1. ¿Cuál fue el tiempo total invertido en el laboratorio por cada uno de ustedes? (Horas/Hombre)

Realicé el laboratorio de forma individual. El tiempo total invertido fue de aproximadamente 2 horas y media, repartidas principalmente en entender el uso de findall, setof y bagof, y en la parte final resolviendo los problemas de búsqueda con DFS (la rana y Batman vs villanos), que fue lo que más tiempo me tomó.

2. ¿Cuál es el estado actual del laboratorio? ¿Por qué?

El laboratorio quedó completo, con todos los ejercicios funcionando correctamente. Logré resolver desde las consultas básicas con findall hasta los problemas más complejos de búsqueda en espacio de estados con DFS, verificando cada solución con las consultas de prueba correspondientes.

3. ¿Cuál considera fue el mayor logro? ¿Por qué?

Para mí el mayor logro fue entender bien la diferencia entre findall, setof y bagof, porque al principio los tres parecían hacer lo mismo, pero cuando trabajé con listas que tenían valores repetidos (como los dueños con varias mascotas) me di cuenta de que cada uno maneja los duplicados y el agrupamiento de forma distinta. Eso me ayudó mucho a la hora de armar la lógica de los problemas de DFS más adelante.

4. ¿Cuál considera que fue el mayor problema técnico? ¿Qué hicieron para resolverlo?

El mayor problema técnico fue armar la función `siguiente_estado/2` en los problemas de búsqueda, sobre todo el de Batman vs villanos, porque había que combinar varias condiciones a la vez (buscar debilidad del villano, buscar el poder correspondiente, verificar el daño y la energía disponible). Para resolverlo fui dividiendo el problema en partes más pequeñas, probando cada condición por separado antes de juntarlas todas en una sola regla, y usando `select/3` para poder ir descontando los villanos ya derrotados de la lista.

5. ¿Qué hicieron bien como equipo? ¿Qué se comprometen a hacer para mejorar los resultados?

Al trabajar solo, lo que hice bien fue organizar el tiempo por partes, resolviendo primero los ejercicios más sencillos (findall, setof, bagof, forall) antes de pasar a los problemas de búsqueda, que eran los más exigentes. Me comprometo a empezar con más anticipación los laboratorios que incluyen problemas de búsqueda (como el de la rana y Batman), porque son los que más tiempo de análisis requieren antes de empezar a codificar.

6. ¿Qué referencias usaron? ¿Cuál fue la más útil? Incluyan citas con estándares adecuados.

Para este laboratorio no consulté muchas fuentes externas, principalmente me apoyé en el material visto en clase. Lo único adicional que revisé fue un video en YouTube para reforzar la

idea de cómo funciona el backtracking en Prolog, lo cual me ayudó a entender mejor cómo se comportan predicados como `bagof/3` y el algoritmo de DFS en los problemas de búsqueda.

MyCyberAcademy. (2013). *Tutorial Prolog - 8 - Backtracking* [Video]. YouTube.
https://www.youtube.com/watch?v=KVv_t28t1QI